

From MCTS to AlphaZero: Summary Report

1. Landscape

For four decades, the strongest game-playing programs — Deep Blue in chess, Stockfish more recently, Elmo in shogi — were built the same way: a human-tuned evaluation function (piece values, king safety, pawn structure, mobility) scored positions, and a heavily engineered alpha-beta search explored the game tree using domain-specific pruning, extensions, and move-ordering heuristics. This machinery worked, but only because grandmasters and engineers had spent decades encoding what "a good position" looks like. None of it transferred to a new domain without starting over.

A separate line of work, culminating in AlphaGo Zero, showed that a program could reach superhuman strength in Go using nothing but the rules of the game and self-play reinforcement learning, replacing the evaluation function with a trained neural network and replacing hand-tuned search heuristics with Monte Carlo Tree Search (MCTS) guided by that network. AlphaZero is the generalization of that idea: one algorithm, no game-specific tuning beyond the rules themselves, applied unmodified to chess, shogi, and Go.

2. Problem

Two things stood in the way of reusing the AlphaGo Zero recipe directly. First, Go's rules are symmetric (rotation/reflection invariant) and binary in outcome (win or loss), which AlphaGo Zero exploited for data augmentation and for a simple win-probability objective. Chess and shogi have neither property: pawns move differently forward than backward, castling is asymmetric, and draws are common (indeed, optimal chess is conjectured to be a draw). Second, the whole point was to see whether a *single* algorithm — not a Go algorithm with patches — could bootstrap expert-level play in a domain with none of Go's convenient structure, purely by having a neural network and a search procedure repeatedly improve each other. The problem, then, was to design a self-play training loop and a search procedure that

- 1) needs no rollout policy, no handcrafted features, and no symmetry assumptions, and
- 2) turns raw search into training signal that makes the network better, so that the improved network makes the search better still.

3. Solution: Building AlphaZero on top of plain MCTS

The cleanest way to understand AlphaZero is to start from vanilla MCTS with UCT and add exactly two ingredients: a neural network that replaces random rollouts, and a modified selection rule (PUCT) that uses the network's opinion instead of pure visit statistics.

3.1 Where classic MCTS/UCT starts

Standard MCTS builds a search tree incrementally through repeated simulations, each with four phases: **select** a path from the root to a leaf using a tree policy; **expand** the leaf by adding one or more children; **evaluate/simulate** the new leaf, classically by playing out a *random rollout* to a terminal state and taking the outcome as an estimate; and **backpropagate** that outcome up the path, incrementing each edge's visit count $N(s,a)$ and updating its average value $Q(s,a)$.

The classic tree policy is UCT (Upper Confidence bound for Trees):

$$UCT(s,a) = Q(s,a) + c \cdot \sqrt{\ln N(s) / N(s,a)}$$

$Q(s,a)$ is the empirical mean outcome of simulations that passed through action a at state s ; the second term is an exploration bonus that shrinks as an action is tried more often ($N(s,a)$ grows) but grows with the log of the parent's total visit count $N(s)$, guaranteeing every action is eventually explored while spending most simulations on the currently-best one. The critical weakness for a domain like chess is that $Q(s,a)$ is meaningless until many random rollouts have been played through it — random rollouts in chess are close to noise, since a single bad or good move played randomly for the rest of the game says very little about the true value of the position.

3.2 The first ingredient: replace rollouts with a network — PUCT instead of UCT

AlphaZero removes random rollouts entirely and replaces both the rollout policy and the rollout-based evaluation with a single deep neural network with two heads:

$$(p, v) = f_{\theta}(s)$$

Given a board position s (encoded, see 3.3), the network outputs a probability vector p over moves — an estimate of which moves are worth exploring, i.e., the "actor" — and a scalar $v \in [-1, 1]$ estimating the expected game outcome from s for the player to move, i.e., the "critic." This is exactly the actor-critic structure, except actor and critic share the same trunk of convolutional layers and are trained jointly.

Because the network now supplies an informed *prior* over actions before any simulation has passed through them, the selection rule is changed from UCT to PUCT (Predictor + UCT):

$$a^* = \operatorname{argmax}_a [Q(s,a) + U(s,a)]$$

$$U(s,a) = c_{\text{puct}} \cdot P(s,a) \cdot \sqrt{ \sum_b N(s,b) } / (1 + N(s,a))$$

Here $P(s,a)$ is the prior probability the network assigned to move a (the policy head's output for that move), and the sum in the numerator is the parent's total visit count, matching UCT's $N(s)$ term. Two differences from UCT matter conceptually:

- **The exploration bonus is weighted by the network's prior**, not left uniform. A move the network considers implausible gets a small bonus even if it has never been tried; a move the network likes gets explored aggressively from the very first simulations. This is what lets AlphaZero search intelligently *before* it has any visit statistics — exactly the gap random rollouts could not fill in tactical domains.
- **The denominator is $(1 + N(s,a))$ rather than $\sqrt{N(s,a)}$** , which decays the prior's influence faster as visits accumulate, so as a subtree is explored more, the search increasingly trusts its own accumulated $Q(s,a)$ over the initial prior — the network's guess is a starting point that gets overridden by evidence, not a permanent bias.

In short: UCT explores by optimism under uncertainty using only the tree's own frequency statistics; PUCT explores by optimism under uncertainty using statistics *plus* a learned belief about which branches are promising, so it needs far fewer simulations to find the same quality of move. This is why AlphaZero can be competitive while examining on the order of 80,000 positions per second in chess versus Stockfish's 70 million — the search is narrower and more selective, "more human-like," because the network does the pruning that alpha-beta's handcrafted heuristics used to do.

3.3 What the leaf evaluation actually is

When selection reaches a leaf s_L (a position not yet in the tree), the network is run once: $f_\theta(s_L) \rightarrow (p, v)$. The leaf is expanded with one child per legal action a , each initialized with prior $P(s_L, a)$ taken from p (illegal moves are masked to zero probability and the remaining probabilities renormalized). The scalar v is then backed up exactly the way a rollout's terminal outcome would have been in classic MCTS — added into the running average $Q(s,a)$ for every edge on the path back to the root, and every $N(s,a)$ along that path incremented by one. So the network's value head is literally standing in for "play this out randomly and see who wins"; it is a learned, one-shot substitute for simulation to the end of the game.

The board itself is represented as a stack of binary/real-valued planes — one-hot indicators for where each piece type sits, repeated for the last 8 half-moves of history (to let the network infer repetition), plus constant planes for whose turn it is, move count, castling rights, and no-progress count. There is no other input: no evaluation features, no domain heuristics beyond what is needed to state the rules. The policy head mirrors this simplicity on the output side — for chess it is an $8 \times 8 \times 73$ stack of planes, where each of the 64 squares names a piece to "pick up," and the 73 planes encode the destination as a compass direction and distance (for queen-like slides), a knight-move shape, or an underpromotion choice. It is a big, structured multi-class softmax over roughly 4,672 candidate moves, masked down to whatever is legal in the current position.

3.4 One MCTS search = one policy-improvement step

Running many simulations (800 per move during training) from the root does something specific and important: it does not just pick a good move, it *computes a better policy than the raw network's p* . Because search concentrates visits on lines the network already likes and then corrects them using backed-up values from deeper, more accurate evaluations, the resulting distribution over root moves — call it π , defined from the root's visit counts $N(\text{root}, a)$, either proportional to N or greedy — is consistently a stronger policy than p alone. This is the crux of the whole method: **MCTS is used as a policy-improvement operator**, turning a mediocre p into a better π using computation, the same way one round of value iteration turns a mediocre value function into a better one.

3.5 The second ingredient: turn every game into training data

Self-play generates the network's own training set. Both sides in a game are played by running this search at every move and sampling (or, at evaluation time, greedily choosing) a move from π . The game continues until the rules declare a terminal result, or a maximum length is reached and the game is scored as a draw. This yields, for every position s_t actually visited in the game, a triple:

$$(s_t, \pi_t, z)$$

where π_t is the improved search policy computed at that move, and $z \in \{-1, 0, +1\}$ is the eventual outcome of the whole game, from the perspective of the player to move at s_t — win, draw, or loss (this trinary target, rather than AlphaGo Zero's binary win/loss, is exactly the accommodation chess and shogi's draws require). Thousands of these triples, harvested continuously from many parallel self-play games using the network's latest parameters, form the training data — no human games, no external labels.

3.6 The training loop and the loss function

The network is trained by stochastic gradient descent to make its own two heads agree with what search + game outcome just demonstrated: minimize

$$l(\theta) = (z - v)^2 - \pi^T \log p + c \|\theta\|^2$$

Each term has a distinct job. The first term is a mean-squared-error regression loss: it pulls the value head's prediction v toward the actual game result z , so that over many games $v(s)$ converges to the true expected

outcome from s under optimal-ish play. The second term is a cross-entropy loss between the search-derived policy π (treated as the target distribution) and the raw network policy p : it distills the expensive, many-simulation search decision back into the cheap single-forward-pass policy head, so that next time the network sees a similar position, its *prior* is already closer to what deep search would have found — effectively compressing search into intuition. The third term is standard L2 weight regularization controlled by c , discouraging overfitting to the (still finite, continually refreshed) pool of self-play data.

Training and self-play run concurrently rather than in strict alternating phases: the very latest network parameters are used to generate new self-play games immediately, with no held-out "champion" network and no gating tournament deciding whether to adopt the new weights (a departure from AlphaGo Zero, which only promoted a challenger network after it beat the incumbent in 55% of an evaluation match). This closes a tight loop — better weights $\rightarrow$ better priors $P(s,a)$ and better $v \rightarrow$ stronger search $\rightarrow$ higher-quality π and more accurate z targets $\rightarrow$ better weights again — that is, in effect, generalized policy iteration carried out entirely through self-play, with MCTS supplying the "improvement" step and gradient descent supplying the "evaluation/fitting" step.

Two more simplifications complete the picture. AlphaZero does not augment training data with the eight rotations/reflections AlphaGo Zero used for Go, since chess and shogi positions are not symmetric; and it reuses one set of search hyperparameters across all three games, the sole exception being the scale of Dirichlet exploration noise added to the root's prior (to keep self-play from collapsing onto a narrow set of openings), which is scaled to roughly the typical branching factor of each game.

4. Side Effects and Limitations

Averaging value estimates over a subtree (as MCTS does) rather than computing an explicit minimax (as alpha-beta does) means individual neural-network approximation errors tend to cancel out over many simulations rather than propagating to the root the way a single alpha-beta line's error would — a plausible reason MCTS pairs well with a noisy learned evaluator where classical tree search historically did not. But the approach also depends on very large compute for training (thousands of first-generation TPUs generating self-play games plus dozens more training the network, across many hours per game), and it discards essentially every classical chess-programming technique — opening books, endgame tablebases, quiescence search — any of which could likely improve it further but were deliberately left out to test the pure self-play approach.

5. Overall Results and Performance

Within about 4 hours (300k training steps) AlphaZero's chess network surpassed Stockfish; in shogi it surpassed Elmo in under 2 hours (110k steps). In 100-game matches at 1 minute/move, AlphaZero lost no games to Stockfish (25 wins, 25 draws as White; 3 wins, 47 draws as Black) and lost only 8 of 100 to the stronger Elmo, while examining roughly three orders of magnitude fewer positions per second than either handcrafted opponent — evidence that a learned, PUCT-guided search directed by a jointly-trained policy-value network can substitute for decades of domain-specific engineering, given only the rules of the game and enough self-play.